NAME

zbash - high-performance silicon command language interpreter and interactive shell

SYNOPSIS

zbash [**-c** *string*] [**-i**] [**-s**] [**--version**] [**--help**] [*file* [*argument* ...]]

DESCRIPTION

zbash is a modern, bare-metal command language interpreter and interactive shell engineered specifically for Apple Silicon and POSIX systems. It combines a hardware-accelerated SIMD vector lexer, a zero-copy pipeline execution engine, and an ergonomic interactive environment featuring dynamic multi-line prompting, real-time repository inspection, automated directory stack tracking, and lifecycle event hooks.

When invoked with standard input attached to a terminal, **zbash** runs in interactive mode, providing advanced line editing, persistent command history synchronization, and visual status indicators. When invoked with a script file or via a pipe, **zbash** executes non-interactively with maximum throughput and zero overhead.

OPTIONS

-c *string*

Commands are read from *string*. If there are arguments after the string, they are assigned to the positional parameters, starting with *$0*.

-i Force the shell to run in interactive mode.

-s Read commands from the standard input. Positional parameters are set to any remaining arguments.

--help Display a summary of available command-line options and exit.

--version

Display shell version, architecture, and copyright information, then exit.

INVOCATION AND STARTUP FILES

Upon initialization, **zbash** determines whether it is executing interactively or as a login shell:

/etc/zprofile Read by login shells before user configuration files.

~/.zprofile Read by interactive login shells.

~/.zbashrc Read by interactive shells on startup. This is the primary location for user aliases, environment definitions, and hook functions.

~/.zbash_history The persistent history file where interactive commands are stored and synchronized across terminal sessions.

INTERACTIVE ENVIRONMENT

zbash provides a rich, responsive interactive terminal experience designed for modern developer ergonomics:

Two-Line Visual Hierarchy Prompt

By default, interactive sessions render a two-line structured prompt:

<?><?> zbash in ~/projects/engine on git:(main)*
<?><?><?> command

- **Context Bar (Top Line):** Displays the active shell name, current working directory (compacted with '~' or path ellipses), and active source control information.
- **Input Indicator (Bottom Line):** Displays an Emerald arrow ('<?><?><?>') upon successful command completion (*$?* = 0), or a Ruby red arrow displaying the non-zero exit code (e.g. '[127]') when a command fails.
- **Native Repository Inspection:** Inspects repository branches and working tree status directly in memory without spawning external child processes, ensuring instantaneous prompt rendering.
- **Execution Duration:** Commands requiring measurable wall-clock time record their execution duration in milliseconds, exported to the environment variable *ZBASH_CMD_DURATION*.

MONACO-GRADE SILICON INTELLISENSE

zbash revolutionizes 40 years of Unix shell interaction by embedding an IDE-grade Monaco language intelligence engine directly into its bare-metal C binary, operating at sub-50 nanosecond latency without any external Node, Python, or LSP background processes:

Real-Time Syntax Highlighting

Every keystroke is lexically parsed and styled using vibrant ANSI 256 colors:

- **Valid System Executables (Neon Emerald):** Commands verified within the O(1) PATH cache render in bold green.
- **Shell Builtins (Electric Aqua):** Internal shell builtins render in bold cyan.
- **Shell Functions (Ice Blue):** User-defined subroutines render in light blue.

- **Invalid Commands (Ruby Red Underline):** Unrecognized command words are immediately underlined in red to catch typos before execution.

- **Flags and Options (Amber Gold):** Command flags (such as **-l** or **--all**) render in gold.

- **Quoted Strings (Warm Orange):** Balanced single and double quoted text strings render in orange, while unclosed quotes display an alert underline.

- **Variables and Expansions (Sky Blue):** Environment variables like *$USER* and *${HOME}* render in sky blue.

- **Operators and Pipes (Neon Magenta):** Process control operators (‘|, ‘&&, ‘||, ‘;, ‘>’’’’’) render in magenta.

Flag Introspection and Live Parameter Ribbon
Directly beneath the prompt, a dynamic status ribbon displays contextual parameter guidance:

- **Synopsis Introspection:** Typing a command displays its synopsis and available arguments.

- **Cluster Flag Decomposition:** Typing clustered short options (e.g. **-czvf**) decomposes all flags simultaneously into individual descriptions.

- **Cross-Pipeline Isolation:** When entering commands within pipelines, the ribbon isolates and inspects the command currently at the cursor.

Floating Monaco Hover Help Box
Pressing **Alt-H**, **Esc-h**, or **F1** renders a floating terminal window with rounded borders, presenting instant on-screen documentation and manual summaries via a sub-millisecond JIT manpage parser.

Autocomplete, Ghost Text, and Enter-Accept Paradigm
zbash provides sub-microsecond inline completion across builtins, executables, shell functions, aliases, flags, and filesystem paths:

- **Inline Ghost Text:** Presents predictive suffixes in subtle titanium gray.

- **Enter-Accept Paradigm:** Pressing **Enter** while ghost text is active automatically absorbs the recommendation into the execution buffer and executes the complete command with zero screen residue, guaranteeing that what the user sees is what executes.

- **Circular Inline Tab Cycling:** Pressing **Tab** successively cycles through all matching candidates in a

round-robin rotation, formatting the live candidate list on the ribbon. Pressing **Esc** cancels the cycle and reverts the input buffer to the original prefix.

- **History Quality Sanitizer:** Commands from previous sessions that failed with exit status 127 are automatically excluded from ghost suggestions to prevent recurring typo pollution.

- **Trailing Space Suppression:** Typing a space after a word suppresses prefix ghost text, allowing direct execution of base commands.

Line Editing and Keybindings

Interactive input uses native terminal line editing supporting standard editing modes:

- **Enter:** Executes the command buffer, automatically absorbing any active ghost text if at line end.

- **Tab:** Cycles through autocomplete candidates in a circular round-robin loop.

- **Esc:** Reverts active tab cycling to the original prefix.

- **Alt-H / Esc-h / F1:** Opens the Monaco floating hover help window.

- **Right Arrow:** Accepts inline ghost text into the editing buffer.

- **Emacs Mode (Default):** Provides standard navigation shortcuts including **Ctrl-A** (beginning of line), **Ctrl-E** (end of line), **Ctrl-K** (kill to end of line), **Ctrl-U** (kill entire line), **Ctrl-W** (kill preceding word), **Alt-F** (forward word), **Alt-B** (backward word), and **Ctrl-R** (reverse incremental history search).

- **Vi Mode:** Enabled via **set -o vi** for modal command and insert editing.

LIFECYCLE EVENT HOOKS

zbash features native lifecycle hook functions that allow users to intercept and execute custom code during key shell events without wrapping external tools:

precmd Executed automatically immediately before each prompt is rendered. Ideal for updating terminal window titles or setting dynamic status indicators.

```
precmd() {
   print -n "\033]0;${PWD}\007"
}
```

preexec

Executed immediately after a command is entered and before execution begins. The full command string is passed as the first parameter (*$1*).

```
preexec() {
   echo "[executing: $1]"
}
```

chpwd Executed automatically whenever the current working directory changes via **cd**, **pushd**, or **popd**.

```
chpwd() {
   ls -la
}
```

DIRECTORY STACK AND NAVIGATION

In addition to standard directory changes, **zbash** maintains an in-memory directory history stack:

cd [**-L** | **-P** [*dir*]]

Changes the current working directory to *dir*. If *dir* is omitted, defaults to the user's home directory. If *dir* is '-', switches to *$OLDPWD*. Respects the colon-separated search path defined in *$CDPATH*.

pushd [*dir* | *+N* | *-N* [**-n**]]

Pushes the target directory onto the directory stack *DIRSTACK*, or rotates stack entries.

popd [+N | -N [**-n**]]

Removes entries from the top or specified offset of the directory stack.

dirs [**-c** | **-l** | **-p** | **-v**]

Displays the contents of the directory stack. With **-v**, displays entries numbered on separate lines.

DIRSTACK

An indexed array containing the current directory stack, with *${DIRSTACK[0]}* representing the active working directory.

COMMAND HISTORY AND MULTI-SESSION SYNCHRONIZATION

zbash manages command history with instant synchronization:

- **Real-Time Persistence:** Executed commands are immediately appended to *~/.zbash_history*, making command history instantaneously available to concurrent terminal windows and tabs.

- **Duplicate Suppression:** Unique history tracking prevents repetitive consecutive commands from

cluttering history storage.

- **History Control Variables:**

HISTSIZE Maximum number of command lines retained in session memory (default: 2000).

HISTFILE Path to the persistent history file.

HISTCONTROL
Colon-separated flags such as ‘ignoredups’ (ignore duplicate lines) and ‘ignorespace’ (ignore lines starting with whitespace).

HISTIGNORE Colon-separated glob patterns defining commands to exclude from history recording.

PARAMETERS AND ARRAYS

zbash supports standard shell variables, indexed arrays, and associative hash maps:

Indexed Arrays

Indexed arrays are zero-based (*${arr[0]}*):

```
fruits=(apple banana cherry)
echo "${fruits[0]}"      # Outputs: apple
echo "${fruits[@]}"      # Outputs all elements
echo "${#fruits[@]}"     # Outputs array length (3)
```

Associative Arrays

Associative arrays (hash maps) are declared using **declare -A** or **typeset -A**:

```
declare -A user_roles
user_roles[alice]="admin"
user_roles[bob]="engineer"
echo "${user_roles[alice]}"  # Outputs: admin
```

Special Parameters

$?
Exit status of the most recently executed foreground pipeline.

$#
Number of positional parameters passed to the current shell or function.

$$
Process ID of the active shell instance.

$!
Process ID of the most recently started background job.

$0
Name of the shell or script currently executing.

*$@, $**
Expands to the positional parameters. When double-quoted, '$@' expands each parameter as a distinct word.

BUILTIN COMMANDS

zbash includes a comprehensive suite of high-performance builtins:

alias [*name[=value]* ...]
Defines or prints command aliases.

builtin *builtin-name* [*arg* ...]
Executes a shell builtin directly, bypassing shell functions with the same name.

caller [*expr*]
Returns the context of any active subroutine call in the execution frame stack.

cd [**-L** | **-P** [*dir*]]
Changes working directory.

command [**-p** [**-v** | **-V** *command* [*arg* ...]]]
Runs a command bypassing shell function lookups.

declare [**-a** | **-A** | **-i** | **-l** | **-r** | **-x** | **-p** [*name[=val]* ...]]
Declares shell variables, attributes, indexed arrays, or associative hash maps.

dirs [**-c** | **-l** | **-p** | **-v**]
Displays directory stack entries.

disown [**-h** | **-a** | **-r** [*jobspec* ...]]
Removes jobs from active job table management.

echo [**-n** | **-e** | **-E** [*string* ...]]
Outputs strings to standard output with optional escape sequence interpretation.

eval [*arg* ...]
Concatenates arguments and evaluates them as a shell command.

exec [**-cl** [**-a** *name* [*command* [*arg* ...]]]]
Replaces the active shell process with the specified command.

exit [*n*] Terminates the shell with exit code *n*.

export [**-n** | **-f** | **-p** [*name[=val]* ...]]
Marks variables for automatic export to the environments of child processes.

fc [**-e** *ename* [**-nlr** [*first* [*last*]]]]
Selects and re-executes or edits commands from the history list.

fg [*jobspec*, **bg** [*jobspec* ...]]
Moves jobs between foreground and background execution.

help [**-s** [*pattern*]]
Displays native contextual documentation and synopses for builtin commands.

history [**-c** | **-d** *offset* | **-awrn** [*filename*]]
Displays or manipulates the active command history list.

jobs [**-l** | **-p** | **-n** | **-r** | **-s** [*jobspec* ...]]
Lists active background jobs and their state.

kill [**-s** *sigspec* | **-n** *signum* | **-***sigspec pid* | *jobspec* ...]
Sends signals to processes or background jobs.

let *expr* [*expr* ...]
Evaluates arithmetic expressions.

local [*name[=val]* ...]
Declares variables scoped locally within the active function frame.

popd [+N | -N [**-n**]]
Removes entries from the directory stack.

printf *format* [*arguments* ...]
Formats and prints arguments according to format specification strings.

pushd [*dir* | *+N* | *-N* [**-n**]]
Pushes a directory onto the directory stack.

pwd [**-L** | **-P**]
Prints the absolute path of the current working directory.

read [**-r** [**-p** *prompt* [**-t** *timeout* [**-a** *array* [**-n** *nchars* [**-s** [*name* ...]]]]]]]
Reads a line from standard input into shell variables.

readonly [**-p** | **-a** | **-A** [*name[=val]* ...]]
Marks variables or arrays as read-only.

return [*n*]
Exits a shell function with return status *n*.

set [**-e** | **-u** | **-x** | **-v** | **-o** *option* [**--** *arg* ...]]
Configures shell environment options and positional parameters.

shift [*n*]
Shifts positional parameters left by *n* positions.

shopt [**-s** | **-u** [**-p** | **-q** [*optname* ...]]]
Toggles optional shell behavior settings.

source *filename* [*arguments* ...]
Reads and executes commands from *filename* in the current shell context.

test *expr*

[*expr* **]** Evaluates conditional expressions.

[[*expression* **]]**
Evaluates advanced conditional expressions with pattern comparison.

trap [**-l** | **-p** [*action sigspec* ...]]
Defines signal handlers and termination traps.

typeset Identical to **declare**.

unalias [**-a** [*name* ...]]
Removes command aliases.

unset [**-v** | **-f** [*name* ...]]
Unsets shell variables or functions.

wait [*pid* | *jobspec* ...]
Waits for background processes or jobs to terminate and returns their exit code.

ENVIRONMENT

CDPATH The search path for the **cd** builtin.

HOME The default directory for the **cd** builtin.

OLDPWD The previous working directory set by the **cd** builtin.

PATH The search path for executable programs.

PWD The current working directory.

ZBASH_CMD_DURATION
The duration of the last foreground command in milliseconds.

FILES

/etc/zprofile
Global system initialization file for login shells.

~/.zprofile User initialization file for login shells.

~/.zbashrc User initialization file for interactive shells.

~/.zbash_history
Default persistent history file.

STANDARDS

zbash complies with POSIX.1-2017 (IEEE Std 1003.1-2017) shell language specifications while providing high-velocity silicon optimizations and modern interactive developer ergonomics.

AUTHORS

zbash was designed and engineered by Wirot Chookeaw <chin6700x>.

COPYRIGHT

Copyright (C) 2026 Wirot Chookeaw. Licensed under the Apache License, Version 2.0.